

# Torque Auto Advisor

*Database, Backup & Migration Plan — Internal Application*

This is the complete infrastructure, backup, security, and future migration plan for Torque Auto Advisor's internal application. The app is currently in development for roughly 30–40 employees. There is no need to migrate away from Supabase now — the priority is structuring the system correctly so a later migration is straightforward and the company never becomes dependent on a single personal account.

## 1. Where You Are Today

---

The current setup is essentially a single chain of ownership:

```
Your personal account
  -> Supabase
      -> Database
      -> Authentication
      -> Storage / documents
  -> Git / GitHub
      -> Internal mobile app
          -> ~30-40 employees
```

The data involved may include:

- Employee name / contact information
- Aadhaar
- PAN
- Other identity / authentication documents
- Professional information
- Designation
- Department
- Sales targets
- Achievement / target performance
- Employee activity / work data
- Login / authentication information
- Potentially salary / HR-related information

### Recommendation today

Don't migrate yet. Continue development in the existing Supabase project — but start preparing the production architecture now.

## 2. The Biggest Change to Plan For

---

Eventually, ownership of the infrastructure needs to shift away from a personal account.



## employees

```
id
employee_code
name
email
phone
department_id
designation_id
status
created_at
updated_at
```

## employee\_documents

```
id
employee_id
document_type
storage_path
uploaded_at
verified_at
```

## targets

```
id
employee_id
target_type
target_value
period
created_at
```

## target\_achievements

```
id
employee_id
target_id
achievement_value
period
created_at
```

This structure makes future migration much easier.

## 5. Aadhaar / PAN Documents Need Special Treatment

---

This is one of the most important parts of the plan. Don't treat Aadhaar/PAN like normal application data.

**Database — store only metadata:**

```
employee_id
document_type
storage_path
```

```
uploaded_at
```

**Storage — the actual document, in a private bucket:**

```
/private/employee-documents/  
  employee123/  
    aadhaar.pdf  
    pan.pdf
```

The bucket must be private, not public. The application should generate controlled, time-limited access when an authorized person needs to view a document.

**Avoid:**

```
https://.../public/aadhaar.pdf
```

...or anything that makes employee documents publicly accessible.

## 6. Backup Strategy — 3-Level System

---

### Level 1 — Supabase's own backups

Use the backup/recovery features available on your Supabase plan. This protects against accidental deletion, bad database changes, application mistakes, database corruption, and the general need to restore an earlier state. This is your primary operational recovery layer.

## 7. Level 2 — Independent Database Backup

---

Don't depend exclusively on Supabase. Keep an independent backup outside the Supabase project.

```
Supabase Production  
  |  
  v  
Automated database dump  
  |  
  v  
Encrypted backup storage
```

Options include AWS S3, Google Cloud Storage, Azure Blob Storage, or another company-controlled secure storage location.

**The important rule: don't keep the only backup inside the same Supabase account/project.**

## 8. Recommended Backup Frequency

---

For a 30–40 person internal application:

| Backup                    | Frequency              | Purpose        |
|---------------------------|------------------------|----------------|
| Supabase automated backup | Daily / plan-dependent | Quick recovery |

| Backup                | Frequency | Purpose             |
|-----------------------|-----------|---------------------|
| Independent DB backup | Daily     | Disaster recovery   |
| Full archive          | Weekly    | Long-term recovery  |
| Monthly archive       | Monthly   | Historical fallback |

This doesn't need to be manual — automate it.

## 9. Back Up the Documents Separately

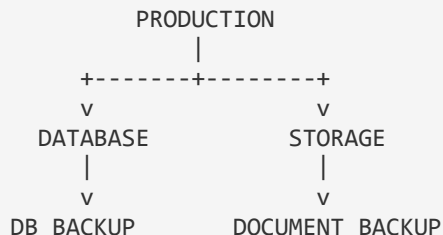
A database backup does not necessarily mean uploaded Aadhaar/PAN files are safely backed up in the same way. Two separate backup systems are needed:

### A. Database backup

Employees  
Targets  
Performance  
Departments  
Authentication-related records  
etc.

### B. Storage backup

Aadhaar  
PAN  
Certificates  
Other employee documents



## 10. Encrypt the Document Backups

Because this involves identity documents, don't simply create a backup.zip and upload it somewhere. The document backup should be encrypted at rest and access controlled — only authorized company administrators should be able to access it.

Do not put employee Aadhaar/PAN files into:

✗ GitHub

- ✗ Git repository
- ✗ Google Drive shared publicly
- ✗ WhatsApp
- ✗ Developer laptops as permanent copies
- ✗ Public Supabase buckets

## 11. GitHub Strategy

---

### The repository should contain:

```
Flutter / React Native / etc. application
API code
Database migrations
Configuration templates
Documentation
```

### It should NOT contain:

```
Aadhaar documents
PAN documents
Employee database dumps
Passwords
Supabase service-role keys
Production secrets
.env files containing real credentials
```

### Use environment variables / secrets instead:

```
Development
-> DEV_SUPABASE_URL
-> DEV_SUPABASE_KEY

Production
-> PROD_SUPABASE_URL
-> PROD_SUPABASE_KEY
```

Never hard-code production secrets into the mobile application.

## 12. Database Migration System

---

Start doing this now. Don't make database changes manually and then forget what changed — use database migrations.

```
Migration 001 Create employees
Migration 002 Create departments
Migration 003 Create targets
Migration 004 Create employee_documents
Migration 005 Add performance fields
```

These migration files live in Git. If a new Supabase project is created in the future, the database structure can be recreated from them — this is what makes migration much easier.

### 13. Future Migration Process

---

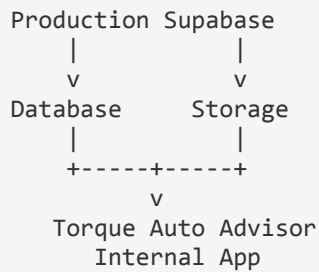
When the company is ready to move off the personal Supabase setup, follow these steps in order:

- Step 1 — Freeze changes:** Temporarily stop database/schema changes.
- Step 2 — Take a complete backup:** Back up database, storage documents, configuration, and database schema.
- Step 3 — Create a company-owned Supabase organization:** e.g. "Torque Auto Advisor".
- Step 4 — Create the production project:** e.g. "Torque Auto Advisor Production".
- Step 5 — Recreate the database schema:** Using the migration files.
- Step 6 — Restore the database data:** Import the production data.
- Step 7 — Restore the documents:** Move the private employee-document storage.
- Step 8 — Configure authentication:** Make sure employee accounts work.
- Step 9 — Configure security policies:** Test that Employee A can access permitted data and cannot access Employee B's sensitive documents.
- Step 10 — Test everything:** Login, employee profile, documents, targets, achievements, reports, admin functions, notifications, permissions, data access.
- Step 11 — Switch the application:** Point the production application to the new project.
- Step 12 — Keep the old project temporarily:** Don't immediately delete it — keep it as a fallback until the migration is confirmed successful.

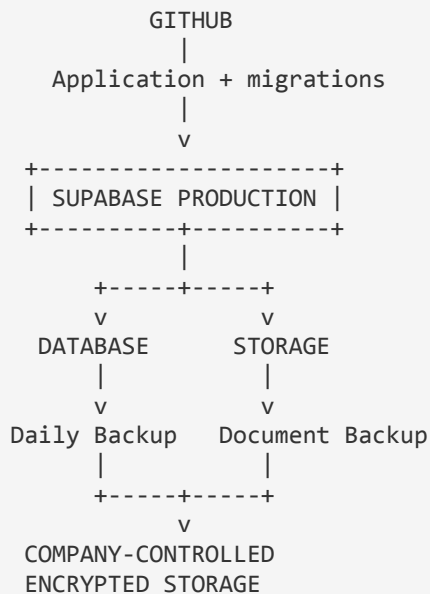
### 14. Migration Diagram

---





## 15. Backup + Migration Architecture



## 16. Access Control

For 30–40 employees, don't give everybody administrative access. Use tiered roles:

```

SUPER ADMIN
  -> HR ADMIN
    -> MANAGER
      -> EMPLOYEE
  
```

**Employee:** Own profile, own targets, own achievements, permitted work information.

**Manager:** Their team's information, team targets, team performance.

**HR:** Manage employee profiles, employee documents, employment information.

**Super Admin:** Manage system, permissions, configuration, security.

This should be enforced at the database / security-policy level (Row Level Security), not only by hiding buttons in the app.

## 17. What You Should NOT Do

---

- ✗ Don't keep everything under your personal account forever
- ✗ Don't put employee documents in GitHub
- ✗ Don't make document storage public
- ✗ Don't manually modify production DB without migrations
- ✗ Don't rely on one backup
- ✗ Don't keep the only backup on your laptop
- ✗ Don't give all 40 employees database-level access
- ✗ Don't put Supabase service-role credentials in the mobile app
- ✗ Don't delete the old project immediately after migration

## 18. The Plan, Right Now

---

You don't need to do everything today — divide it into four phases.

### PHASE 1 — NOW: Development

Continue using the current Supabase project. Do these things:

#### 1. GitHub

- Keep code in Git
- Add database migrations
- Add .gitignore
- Remove secrets from the repository

#### 2. Supabase

- Separate development data from future production data
- Enable appropriate backup/recovery features
- Keep storage buckets private

#### 3. Database

- Proper tables
- Foreign keys
- UUIDs / IDs
- created\_at
- updated\_at
- Migrations

#### 4. Security

- Row Level Security
- Role-based access
- Private document storage

## 19. Phase 2 — Before Real Employee Data

---

Before putting actual Aadhaar/PAN and other sensitive employee information into the system, create company-controlled infrastructure:

```
Company_email
-> Company Supabase organization
-> Production project
```

You remain an administrator/developer. This is the point to stop treating your personal account as the owner of production infrastructure.

## 20. Phase 3 — Production

---

```
Company Supabase
|
|-- Production DB
|-- Private Storage
|-- Authentication
\-- Backups
```

```
GitHub
|
v
CI/CD
|
v
Production
```

Deployments can later be automated as well.

## 21. Phase 4 — Disaster Recovery

---

You should eventually be able to answer: "If our Supabase project disappeared tomorrow, can we recover?" The answer should be yes, because you have:

```
GitHub
+
Database backup
+
Document backup
+
Database migrations
+
Configuration documentation
```

Periodically perform a restore test. A backup that has never been successfully restored is not something to fully trust.

## 22. The Backup Schedule

---

For this size of organization:

| Cadence               | Action                                |
|-----------------------|---------------------------------------|
| Every day             | Automatic database backup             |
| Every day / regularly | Independent encrypted database backup |
| Every week            | Full backup verification              |
| Every month           | Restore test                          |

```

Monday    -> Daily DB backup
Tuesday   -> Daily DB backup
Wednesday -> Daily DB backup
Thursday  -> Daily DB backup
Friday    -> Daily DB backup
Saturday  -> Daily DB backup
Sunday    -> Weekly archive

Monthly   -> Test restoration

```

This can all be automated.

## 23. Retention

---

Don't keep infinite backups. A reasonable starting policy:

| Backup type     | Retention |
|-----------------|-----------|
| Daily backups   | 30 days   |
| Weekly backups  | 3 months  |
| Monthly backups | 12 months |

The exact retention should ultimately follow the company's legal/HR requirements.

## 24. Recommendation Summary

---

**Today:** Don't migrate. Continue development on the existing Supabase project.

**Now:** Implement Git + database migrations + RLS + private storage + backups.

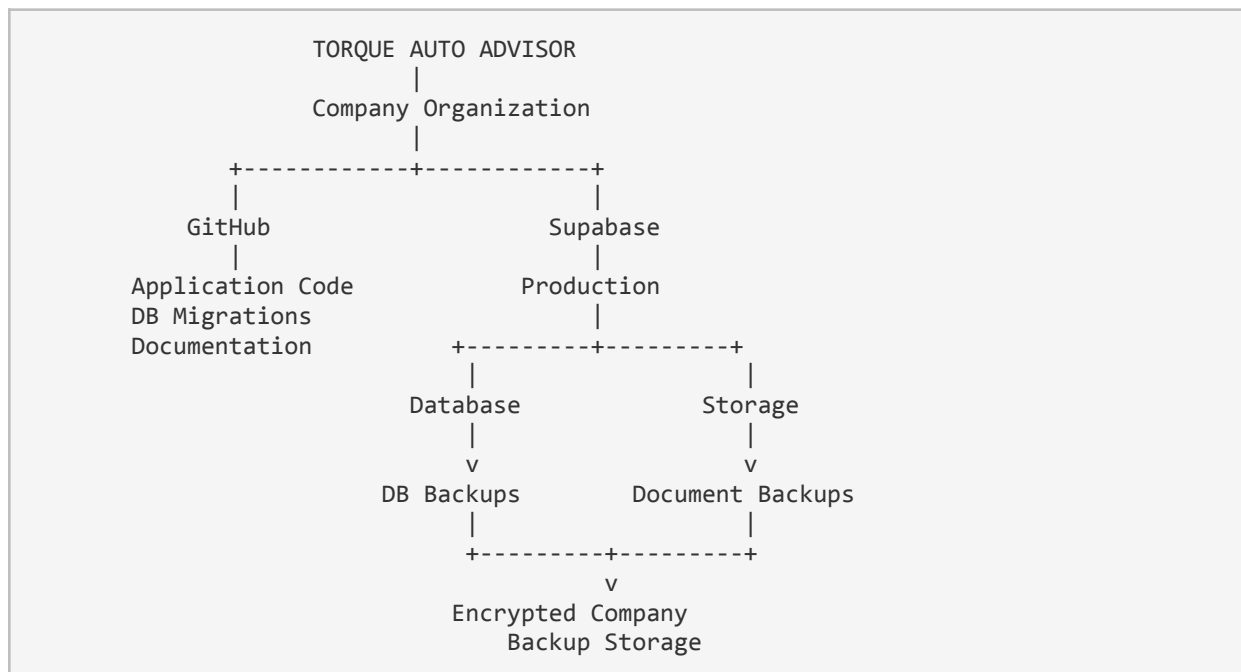
**Before real production:** Move the project/infrastructure to a company-owned Supabase organization/account.

**Before uploading real Aadhaar/PAN data:** Make sure private storage + encryption/access controls + backup + recovery process are properly tested.

**After production:** Have daily automated backup + independent encrypted backup + monthly restore test.

**Later:** If the company eventually outgrows Supabase or needs another architecture, migration is possible because the application will already have a clean database schema, migration files, independent backups, document storage backup, and company ownership. You are not locked into Supabase.

## The Final Architecture



**Bottom line:** *There is no migration problem right now — there is an ownership, backup, and security architecture problem to solve before production. Since the application is still in development, this is the best time to set it up correctly. The existing Supabase project can remain the development environment while the migration/backup foundation is built around it.*